Recap – DM half

Things to cover

- The Master Theorem
- Complexity theory: P, NP and NP-completeness, proving NP-hardness by reductions
- Backtracking, branch-and-bound & linear programming

But first, some general points

Things you can do for revision:

- Look back through lecture slides, referring to the relevant parts of the textbook where necessary
- Do the exercises given in each lecture/other exercises at the end of each textbook chapter
- Last year's exam (except q5)
- Come to the tutorial today and/or next Tuesday, and **ask for help** with things you're struggling with

But first, some general points

For the exam:

- Justify things you say (e.g. if you give a reduction, show that the original instance and the instance you produced have the same answer)
- The justification can be very brief if appropriate! (e.g. “the certificate is the set of objects chosen”)
- Please please please write down partial answers, even if there are things you’re confused about or can’t do – we want to give you marks but the one situation where we can’t is a blank page!

Divide and conquer

- Solve a problem by breaking it into subproblems, solving the subproblems, then combining the solutions to get a solution to the original problem
- E.g. mergesort
- This gives us a cost something like $T(n) = aT(n/b) + f(n)$, where
- a is the number of subproblems
- b is the decrease in the size of each subproblem
- f(n) is the cost of combining the subproblems (often $\Theta(n^c)$but not always)
- E.g. for mergesort: $T(n) = 2T(n/2) + \Theta(n)$

The Master Theorem

- We have T(n) = aT(n/b) + f(n), and we want to find T(n)
- The key question: is f(n) bigger or smaller than $n^{\log_b a}$? (so if f(n) = $\Theta(n^c)$, is c bigger or smaller than $\log_b a$?)
- This tells us whether solving the subproblems is all the work, or the combination is all the work, or they both matter

The Master Theorem

- T(n) = aT(n/b) + f(n)
- Case 1: $f(n) = O(n^{\log_b a - \epsilon})$ (or c < $\log_b$ a)
- This means the subproblems are all that matter
- $T(n) = \Theta(n^{\log_b a})$
- Case 3: $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ (or c > $\log_b$ a), plus regularity condition (don't worry about it)
- This means the recombination work is all that matters
- $T(n) = \Theta(f(n))$

The Master Theorem

- T(n) = aT(n/b) + f(n)
- Case 2: somewhere in between
- $f(n) = \Theta(n^{\log_b a} \log^k n)$ (or c = $\log_b$a)
- Both matter, $T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right)$ (simple case: $n^{\log_b a} \log n$), i.e. ‘gain a factor of log n’.

Examples

- Merge sort: T(n) = 2T(n/2) + Θ(n)
- f(n)=n, comparing to $n^{\log_b a} = n^1 = n$
- They are comparable, so we are in case 2
- $f(n) = \Theta(n^{\log_b a} \log^0 n)$, so $T(n) = \Theta(n \log n)$

Examples

- Karatsuba's algorithm: we can do multiplication with 3 multiplies of half-as-big numbers rather than 4
- T(n)=3T(n/2)+Θ(n)
- f(n)=n, comparing to $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$ (even without a calculator you know $1 < \log_2 3 < 2$!)
- $n^{\log_b a}$ is bigger, so we are in case 1
- So $T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^{\log_2 3}\right) \approx n^{1.585}$

Complexity theory

- Think of problems as decision problems, so the answer is always ‘yes’ or ‘no’, therefore also as languages (set of instances where the answer is yes)
- Most problems can be put in this form (e.g. ‘does G have a [whatever] with cost at most k’?)
- P: can be solved in deterministic polynomial time
- NP: ‘yes’ answers can be verified in deterministic polynomial time

NP

- ‘Yes’ answers can be verified
- Definition 1: there exists a polynomial time verifier B which takes a problem instance x and a ‘certificate’ y, and is *sound* and *complete*
- Sound: if $x \notin L$ then B(x,y) rejects for all y
- Complete: if $x \in L$ then there exists y such that B(x,y) accepts
- Definition 2: there is a non-deterministic algorithm (i.e. can make non-deterministic choices), such that:
- if $x \in L$ then our algorithm can say yes (if we get lucky with the choices)
- If $x \notin L$ then our algorithm will never say yes (no matter the choices)

NP-completeness

- Remark: $P \subseteq NP$ (i.e. every problem in P is in NP)
- We think they're not equal, but we have utterly failed to prove it
- So no chance of proving any NP problem is not in P
- Next best thing: NP-hard, i.e. as hard as any problem in NP (NP-complete=NP-hard and also in NP)

NP-completeness

- To show that a language L is NP-hard, we show that any language L’ in NP can be reduced to it, i.e.
- There is a polynomial time function f which takes instances of L’ and produces instances of L, such that $x \in L' \Leftrightarrow f(x) \in L$
- So if we had a poly time algorithm for L we could get a poly time algorithm for L’
- Or in particular if we believe there’s no poly time algorithm for L’ we have to believe there’s no poly time algorithm for L

NP-completeness

- Fortunately we don’t have to do this for every L’ in NP – it’s enough to do it for a single NP-complete problem L’
- The classic NP-complete problem is 3SAT: satisfiability of CNF formulae where each clause has exactly 3 variables
- Lots of other problems have also been shown to be NP-complete, e.g. Vertex-Cover, Clique, TSP, …

NP-completeness

You should be able to:

- Convert between optimisation and decision version of problems
- Understand the definition(s) of NP, particularly NP1, and show problems are in NP
- Understand what a reduction is, and come up with your own (only simple ones or very similar to the examples seen in lectures/in the assignments)
- Thereby show problems are NP-complete (i.e. NP-hard and in NP)
- Have some familiarity with the problems we saw in lectures.

Backtrack and branch-and-bound

- Backtrack: for decision problems, build up a solution by exhausting variable-by-variable, but cut off search when a partial solution is already infeasible.
- We need a function extend(a,i) to give us a set of partial solutions extending a partial solution a, such that every extension of a is an extension of an element of extend(a,i)
- And a function dead that tells us if we know a partial solution a cannot be extended to a full solution.
- You should be able to write a backtracking algorithm for a particular problem in pseudocode.

Branch-and-bound

- Branch-and-bound: similar for optimisation problems – this time we cut off search when we know a partial solution can’t be extended to a solution that’s better than the best solution we already know
- We need a function bound(a) to give a bound on how good any solution extending a partial solution a can be.
- Again, you should be able to write pseudocode for this.
- How do we get a bound on the ultimate value of a partial solution?
- Can be ad hoc, or can be from linear programming

Linear programming

- Maximise/minimise a linear function subject to linear constraints
- maximise $\sum_{i \in V} c_i x_i$
- subject to
  - $\sum_{j \in V} a_{ij} x_j \leq b_i$ for each $i \in C$
  - $x_i \geq 0$ for each $i \in V$.
- Can be solved in polynomial time (simplex algorithm usually efficient but not technically poly time, interior point methods are)

Integer Linear Programming

- We can write NP problems as linear programming problems, with the extra constraint that the variables are integer-valued.
- For example, Vertex-Cover:
- Minimise $\sum_{v \in V} x_v$
- Subject to $x_v + x_w \geq 1$ for each $vw \in E$
  $0 \leq x_v \leq 1$ for each $v \in V$
- **and x_v an integer for all v**
- Hence, Integer Linear Programming is NP-complete

Integer Linear Programming

- This gives us a way to try to solve NP-complete problems:

1. Write the problem as an ILP
2. Use branch-and-bound to solve the ILP
   - The ‘bound’ part comes from the *fractional relaxation* of the ILP:
   - If we forget the integrality constraint, since we just removed a constraint the optimal value can only get better

Integer Linear Programming

- You should be able to formulate problems (optimisation or decision) as ILP,
- and use the fractional relaxation to make a branch-and-bound algorithm.

Good luck!